

AI Assistant Coding

Assignment 7.5

Name : E.ANJANI

HT. No : 2303A52419

Batch: 32

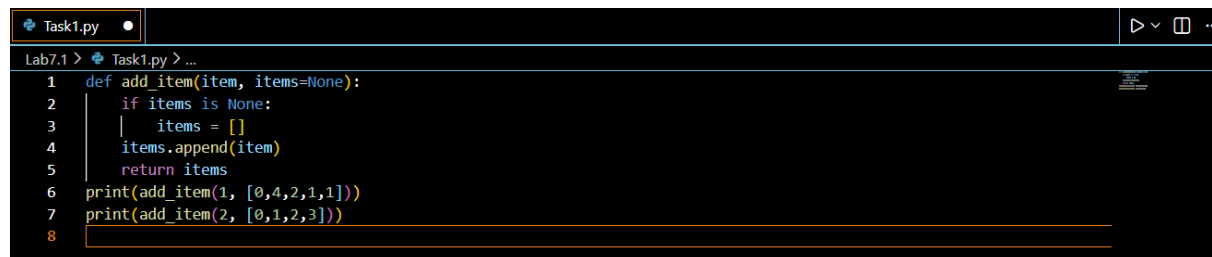
Task 1 (Mutable Default Argument – Function Bug)

Task: Analyze given code where a mutable default argument causes unexpected behavior. Use AI to fix it.

Bug: Mutable default argument

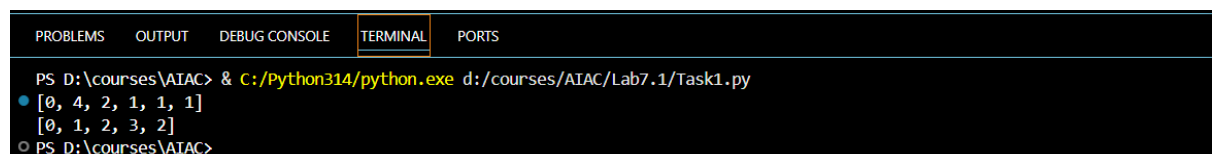
```
def add_item(item, items=[]):  
    items.append(item)  
    return items  
print(add_item(1))  
print(add_item(2))
```

Corrected Code:



```
Task1.py  
Lab7.1 > Task1.py > ...  
1 def add_item(item, items=None):  
2     if items is None:  
3         items = []  
4         items.append(item)  
5     return items  
6 print(add_item(1, [0,4,2,1,1]))  
7 print(add_item(2, [0,1,2,3]))  
8
```

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab7.1/Task1.py  
• [0, 4, 2, 1, 1, 1]  
[0, 1, 2, 3, 2]  
○ PS D:\courses\AIAC>
```

Task 2 (Floating-Point Precision Error)

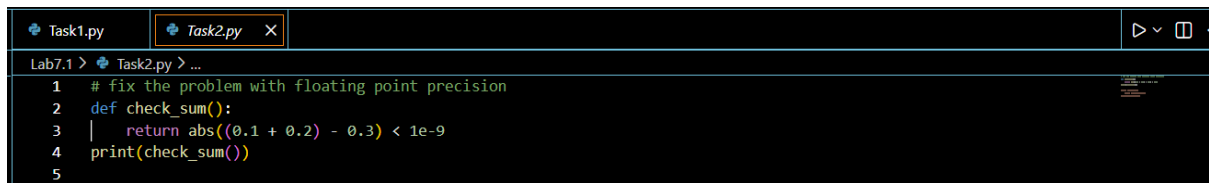
Task: Analyze given code where floating-point comparison fails.

Use AI to correct with tolerance.

Bug: Floating point precision issue

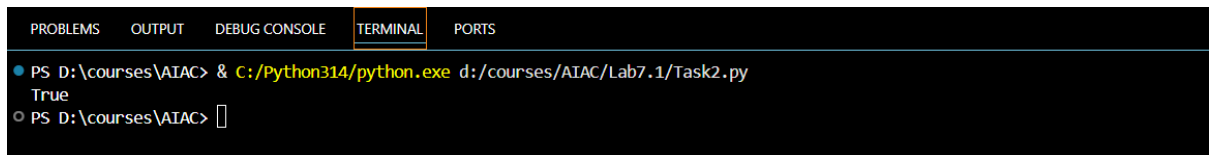
```
def check_sum():  
    return (0.1 + 0.2) == 0.3  
print(check_sum())
```

Corrected Code:



```
Task1.py Task2.py X
Lab7.1 > Task2.py > ...
1 # fix the problem with floating point precision
2 def check_sum():
3     return abs((0.1 + 0.2) - 0.3) < 1e-9
4 print(check_sum())
5
```

Output:

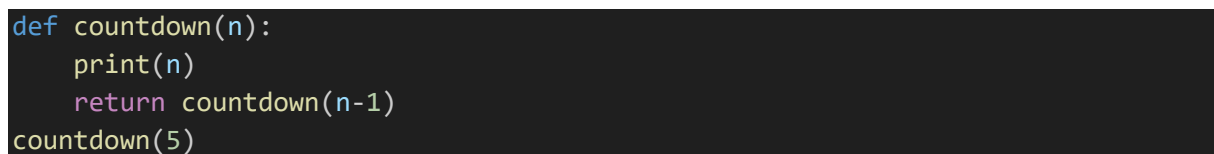


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab7.1/Task2.py
True
PS D:\courses\AIAC>
```

Task 3 (Recursion Error – Missing Base Case)

Task: Analyze given code where recursion runs infinitely due to missing base case. Use AI to fix.

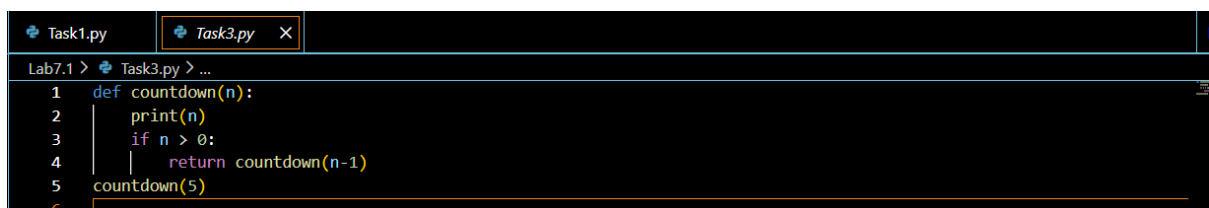
Bug: No base case



```
def countdown(n):
    print(n)
    return countdown(n-1)
countdown(5)
```

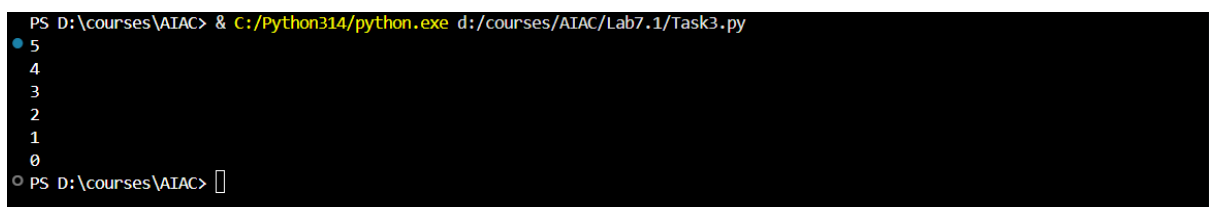
Expected Output : Correct recursion with stopping condition.

Corrected Code:



```
Task1.py Task3.py X
Lab7.1 > Task3.py > ...
1 def countdown(n):
2     print(n)
3     if n > 0:
4         return countdown(n-1)
5 countdown(5)
6
```

Output:



```
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab7.1/Task3.py
5
4
3
2
1
0
PS D:\courses\AIAC>
```

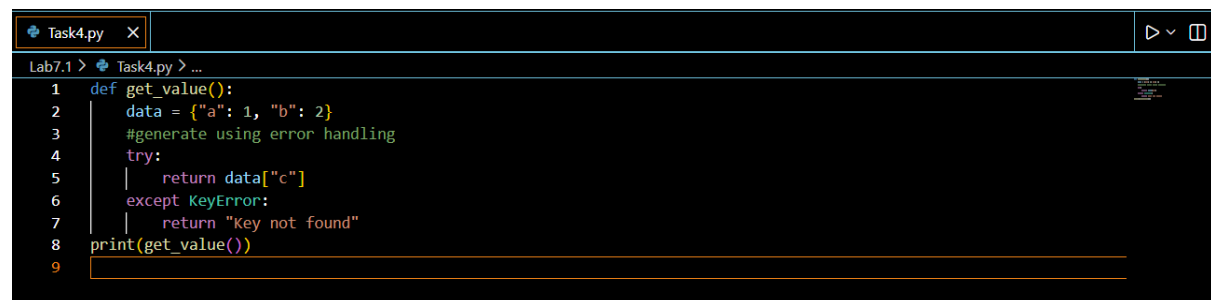
Task 4 (Dictionary Key Error)

Task: Analyze given code where a missing dictionary key causes error. Use AI to fix it.

Bug: Accessing non-existing key

```
def get_value():  
    data = {"a": 1, "b": 2}  
    return data["c"]  
print(get_value())
```

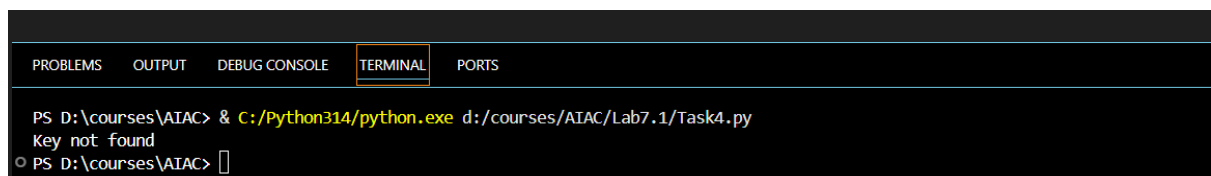
Corrected Code:



The screenshot shows a code editor window titled 'Task4.py'. The code is as follows:

```
1 def get_value():  
2     data = {"a": 1, "b": 2}  
3     #generate using error handling  
4     try:  
5         return data["c"]  
6     except KeyError:  
7         return "Key not found"  
8 print(get_value())  
9
```

Output:



The screenshot shows a terminal window with the following output:

```
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab7.1/Task4.py  
Key not found  
PS D:\courses\AIAC>
```

Task 5 (Infinite Loop – Wrong Condition)

Task: Analyze given code where loop never ends. Use AI to detect and fix it.

Bug: Infinite loop

```
def loop_example():  
    i = 0  
    while i < 5:  
        print(i)
```

Expected Output: Corrected loop increments i.

Corrected Code:

```
Task4.py Task5_.py X
Lab7.1 > Task5_.py > loop_example
1 def loop_example():
2     i = 0
3     while i < 5:
4         print(i)
5         i += 1
6     loop_example()
```

Output:

```
key not found
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab7.1/Task5_.py
0
1
2
3
4
PS D:\courses\AIAC>
```

Task 6 (Unpacking Error – Wrong Variables)

Task: Analyze given code where tuple unpacking fails. Use AI to fix it.

Bug: Wrong unpacking

```
a, b = (1, 2, 3)
```

Expected Output: Correct unpacking or using _ for extra values.

Corrected Code:

```
Lab7.1 > Task6_.py > ...
1 a, b, c = (1, 2, 3)
2 print(a, b, c)
```

Output:

```
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab7.1/Task6_.py
1 2 3
PS D:\courses\AIAC>
```

Task 7 (Mixed Indentation – Tabs vs Spaces)

Task: Analyze given code where mixed indentation breaks execution. Use AI to fix it.

Bug: Mixed indentation

```
def func():
    x = 5
        y = 10
    return x+y
```

Expected Output : Consistent indentation applied.

Corrected Code:

```
Task4.py Task7.py X
Lab7.1 > Task7.py > ...
1 def func():
2     x = 5
3     y = 10
4     return x + y
5 print(func())

print(result)
```

Output:

```
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab7.1/Task7.py
15
PS D:\courses\AIAC>
```

Task 8 (Import Error – Wrong Module Usage)

Task: Analyze given code with incorrect import. Use AI to fix.

Bug: Wrong import

```
import maths
print(maths.sqrt(16))
```

Expected Output: Corrected to import math

Corrected Code:

```
Task8.py X
Lab7.1 > Task8.py
1 import math
2 print(math.sqrt(16))
```

Output:

```
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab7.1/Task8.py
4.0
PS D:\courses\AIAC>
```

Task 9 (Unreachable Code – Return Inside Loop)

Task: Analyze given code where a return inside a loop prevents full iteration. Use AI to fix it.

Bug: Early return inside loop

```
def total(numbers):
    for n in numbers:
        return n
print(total([1,2,3]))
```

Expected Output: Corrected code accumulates sum and returns

after loop.

Corrected Code:

```
Task9.py X
Lab7.1 > Task9.py > total
1 def total(numbers):
2     total = 0
3     for n in numbers:
4         total += n
5     return total
6 print(total([1,2,3]))
```

Output:

```
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab7.1/Task9.py
6
PS D:\courses\AIAC>
```

Task 10 (Name Error – Undefined Variable)

Task: Analyze given code where a variable is used before being defined. Let AI detect and fix the error.

Bug: Using undefined variable

```
def calculate_area():
    return length * width
print(calculate_area())
```

Corrected Code:

```
Task10.py X
Lab7.1 > Task10.py > ...
1 def calculate_area(length, width):
2     return length * width
3 print(calculate_area(5, 10))
4 print(calculate_area(7, 3))
5 print(calculate_area(2.5, 4.5))
6 print(calculate_area(0, 5))
```

Output:

```
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab7.1/Task10.py
50
21
11.25
0
PS D:\courses\AIAC>
```

Explanation:

This code will not work because the function `calculate\_area` is defined without any parameters, but it tries to use the variables `length` and `width` which are not defined within the function. To fix this, you need to define `length` and `width` as parameters of the function so that you can pass values to it when calling the function.

Task 11 (Type Error – Mixing Data Types Incorrectly)

Task: Analyze given code where integers and strings are added incorrectly. Let AI detect and fix the error.

Bug: Adding integer and string

```
def add_values():  
    return 5 + "10"  
print(add_values())
```

Corrected Code:

```
Lab7.1 > Task11.py > ...  
1 def add_values():  
2     return 5 + int("10")  
3 print(add_values())  
4  
5 #Test Cases  
6 assert add_values() == 15, "Test Case 1 Failed"  
7 assert isinstance(add_values(), int), "Test Case 2 Failed"  
8 print("All Test Cases Passed")
```

Output:

```
PS D:\courses\AIAC> & C:\Python314\python.exe d:/courses/AIAC/Lab7.1/Task11.py  
15  
All Test Cases Passed  
PS D:\courses\AIAC> |
```

Explanation:

The code will not work because it is trying to add an integer (5) and a string ("10"), which is not allowed in Python. This will raise a `TypeError`. To fix this, you can either convert the string to an integer or the integer to a string before performing the addition.

Task 12 (Type Error – String + List Concatenation)

Task: Analyze code where a string is incorrectly added to a list.

Bug: Adding string and list

```
def combine():  
    return "Numbers: " + [1, 2, 3]  
print(combine())
```

Corrected Code:

```
Task12.py X
Lab7.1 > Task12.py > ...
1 def combine():
2 |     return "Numbers: " + str([1, 2, 3])
3 print(combine())
4
5 #Test Cases
6 assert combine() == "Numbers: [1, 2, 3]", "Test Case 1 Failed"
7 assert isinstance(combine(), str), "Test Case 2 Failed"
8 print("All Test Cases Passed")
```

Output:

```
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab7.1/Task12.py
Numbers: [1, 2, 3]
All Test Cases Passed
PS D:\courses\AIAC>
```

Explanation:

The code will not work because it is trying to concatenate a string ("Numbers: ") with a list ([1, 2, 3]). In Python, you cannot directly concatenate a string and a list, which will result in a `TypeError`.

Task 13 (Type Error – Multiplying String by Float)

Task: Detect and fix code where a string is multiplied by a float.

Bug: Multiplying string by float

```
def repeat_text():
    return "Hello" * 2.5
print(repeat_text())
```

Corrected Code:

```
Task13.py X
Lab7.1 > Task13.py > ...
1 def repeat_text():
2 |     return "Hello" * int(2.5)
3 print(repeat_text())
4
5 #Test Cases
6 assert repeat_text() == "HelloHello", "Test Case 1 Failed"
7 assert isinstance(repeat_text(), str), "Test Case 2 Failed"
8 print("All Test Cases Passed")
```

Output:

```
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab7.1/Task13.py
HelloHello
All Test Cases Passed
PS D:\courses\AIAC>
```


Explanation:

The code will not work because you cannot multiply a string by a non-integer (in this case, 2.5). In Python, multiplying a string by an integer repeats the string that many times, but multiplying by a float is not allowed and will raise a `TypeError`.

Task 14 (Type Error – Adding None to Integer)

Task: Analyze code where `None` is added to an integer.

Bug: Adding `None` and integer

```
def compute():  
    value = None  
    return value + 10  
print(compute())
```

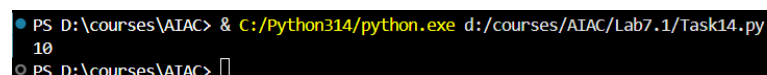
Corrected Code:



The screenshot shows a code editor window titled 'Task14.py'. The code is as follows:

```
1 def compute(value = 0):  
2     return value + 10  
3 print(compute())  
4  
5
```

Output:



The screenshot shows a terminal window with the following output:

```
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab7.1/Task14.py  
10  
PS D:\courses\AIAC>
```

Explanation:

The code will not work because the variable `value` is set to `None`, and you cannot add an integer to `None`. To fix this, you should initialize `value` with a numeric value, such as 0, or modify the function to accept a parameter with a default value.

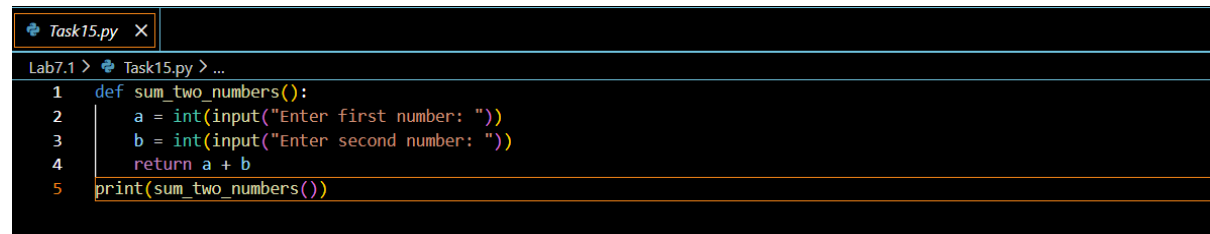
Task 15 (Type Error – Input Treated as String Instead of Number)

Task: Fix code where user input is not converted properly.

Bug: Input remains string

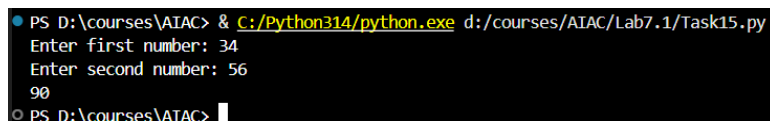
```
def sum_two_numbers():  
    a = input("Enter first number: ")  
    b = input("Enter second number: ")  
    return a + b  
print(sum_two_numbers())
```

Corrected Code:



```
Task15.py X  
Lab7.1 > Task15.py > ...  
1 def sum_two_numbers():  
2     a = int(input("Enter first number: "))  
3     b = int(input("Enter second number: "))  
4     return a + b  
5 print(sum_two_numbers())
```

Output:



```
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab7.1/Task15.py  
Enter first number: 34  
Enter second number: 56  
90  
PS D:\courses\AIAC>
```

Explanation:

This code will not work as intended because the `input()` function in Python always returns a string. When you use the `+` operator with two strings, it performs string concatenation instead of numerical addition. For example, if you enter `3` and `4`, the output will be `34` instead of `7`.